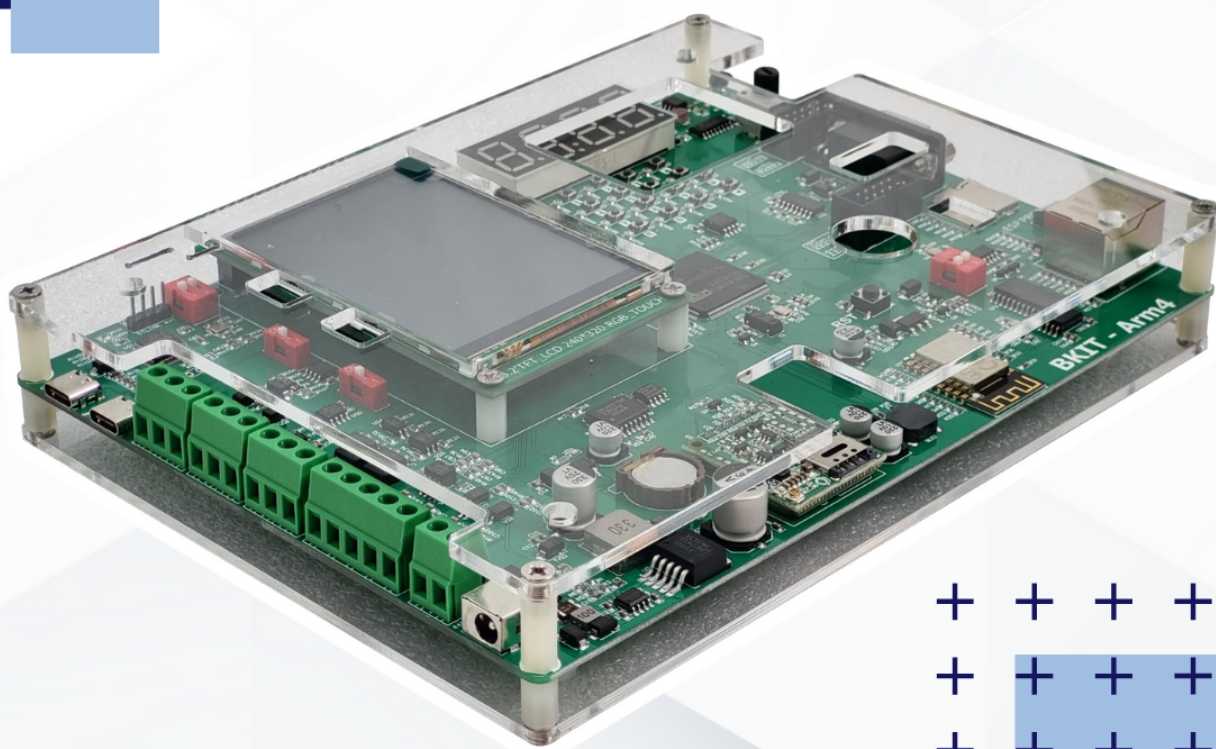


KIT THÍ NGHIỆM BKIT ARM4

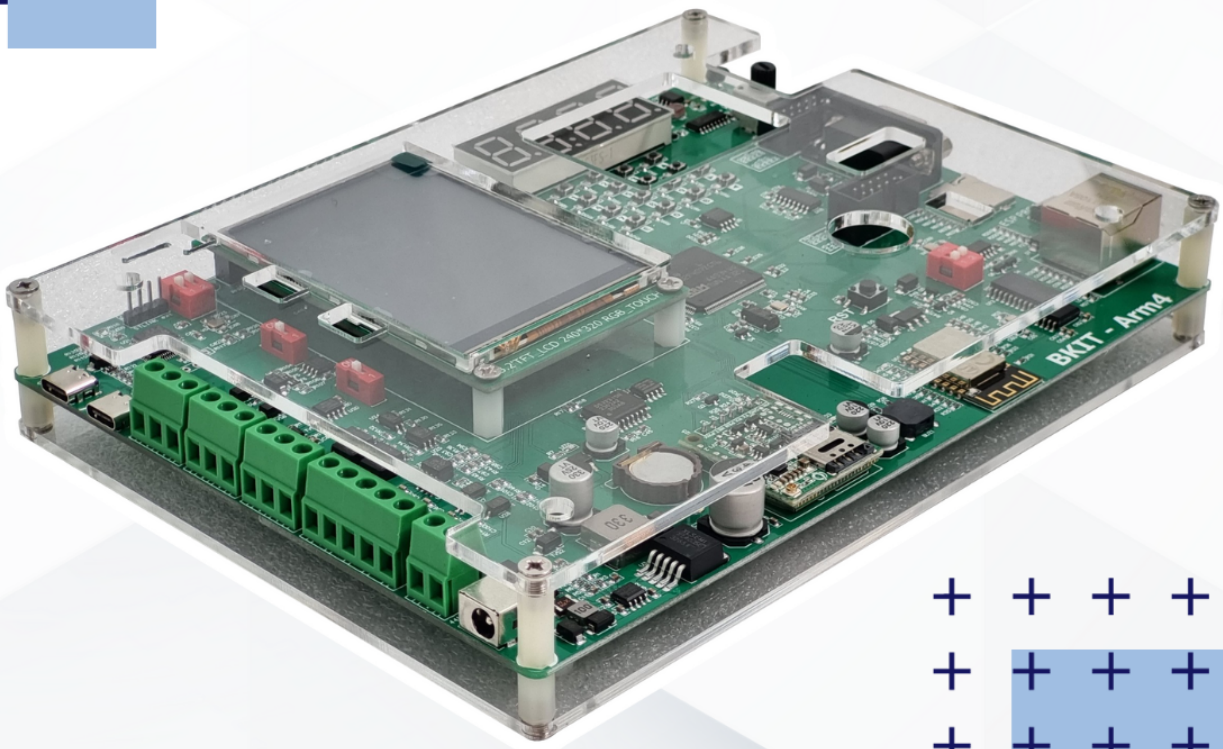


Mục lục

Chapter 2. Timer Interrupt and LED Scanning	3
1 Mục tiêu	4
2 Giới thiệu	4
3 Cấu hình timer	5
4 Cấu hình LED đồng hồ	7
4.1 Giao tiếp SPI	7
4.2 IC 74HC595	8
4.3 Config chân cho vi điều khiển	9
5 Hướng dẫn lập trình	12
6 Bài tập và Báo cáo	17
6.1 Bài tập 1	17
6.2 Bài tập 2	18
6.3 Bài tập 3	18
6.4 Bài tập 4	18
6.5 Bài tập 5	18

CHƯƠNG 2

Timer Interrupt and LED Scanning

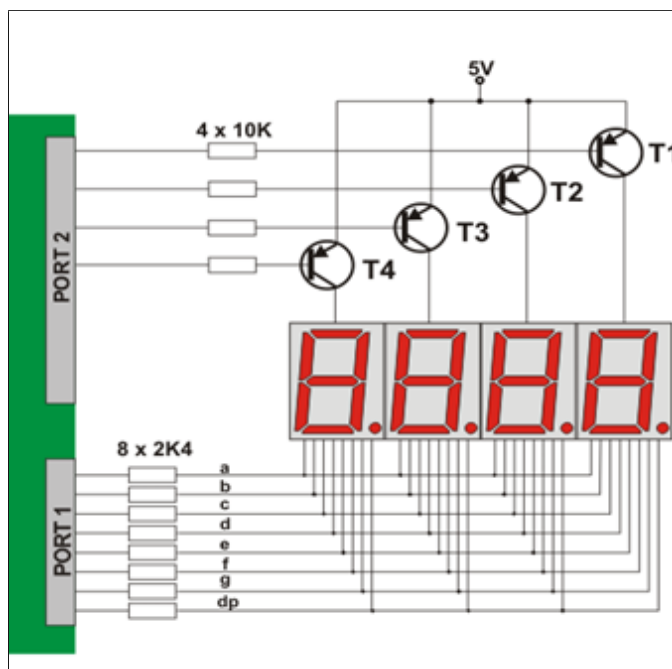


1 Mục tiêu

- Biết cách cấu hình và sử dụng timer trên kit thí nghiệm.
- Biết cách cấu hình và sử dụng thư viện LED bảy đoạn trên kit thí nghiệm.

2 Giới thiệu

Timer là một tính năng quan trọng trong vi điều khiển hiện đại. Chúng giúp đo thời gian thực hiện tác vụ, tạo non-blocking code, kiểm soát pin timing và chạy hệ điều hành. Trong hướng dẫn này, chúng ta sẽ tìm hiểu cách cấu hình và sử dụng timer để chớp tắt đèn LED. Sử dụng timer interrupt để quét LED.

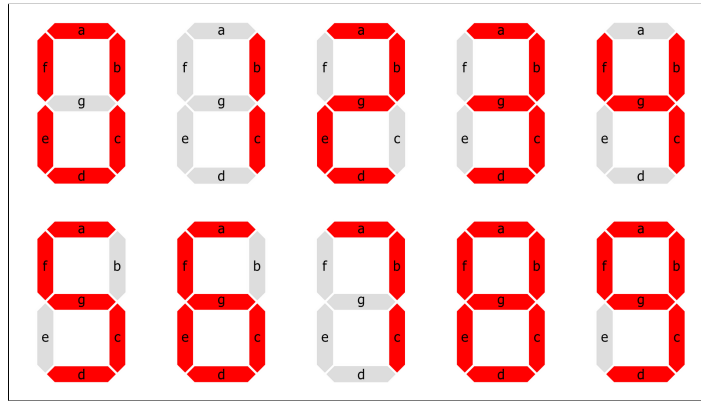


Hình 2.1: Giao diện của 4 LED bảy đoạn kết nối với vi điều khiển

Thiết kế giao diện cho màn hình nhiều LED bảy đoạn sử dụng input và output để điều khiển tất cả các LED trong màn hình đã cho: dòng điện vào, khả năng cấp nguồn hay truyền nguồn cho thiết bị khác của mỗi chân, tốc độ gửi tín hiệu điều khiển. Với các thông số kỹ thuật này, giao tiếp có thể được thực hiện cho màn hình nhiều LED bảy đoạn với một vi điều khiển.

Mỗi LED bảy đoạn có bảy phân đoạn được đánh dấu từ a đến g (hình 2.2) và phân đoạn thứ tám là dấu chấm thập phân. Trong diagram, mỗi LED bảy đoạn có 8 đèn LED bên trong tương ứng với 8 phân đoạn, nên tổng số đèn LED trong ma trận 4 LED bảy đoạn là 32. Tuy nhiên, không cần phải bật tắt tất cả các đèn LED, chỉ cần một

trong số chúng. Do đó, chỉ cần 12 chân để điều khiển 4 LED bảy đoạn. Sử dụng vi điều khiển, ta có thể bật các đèn cùng một khoảng thời gian T_S . Vì vậy, chu kỳ để điều khiển 4 LED bảy đoạn sẽ là $4T_S$. Nói cách khác, những LED này được quét với tần số $f = 1/4T_S$. Cuối cùng, dễ thấy nếu tần số quét lớn hơn 30Hz (e.g. $f = 50\text{Hz}$) thì các LED dường như đang sáng cùng một lúc. Trong bài hướng dẫn này, timer interrupt được sử dụng để thiết kế một khoảng thời gian T_S cho việc quét LED.

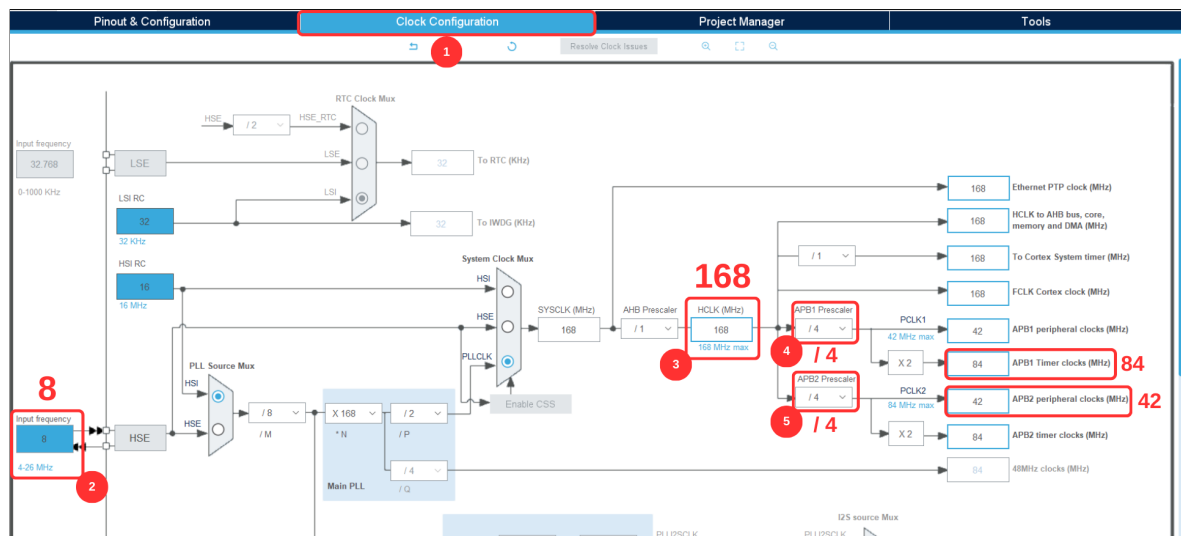


Hình 2.2: Biểu diễn ký tự số bằng LED 7 đoạn

3 Cấu hình timer

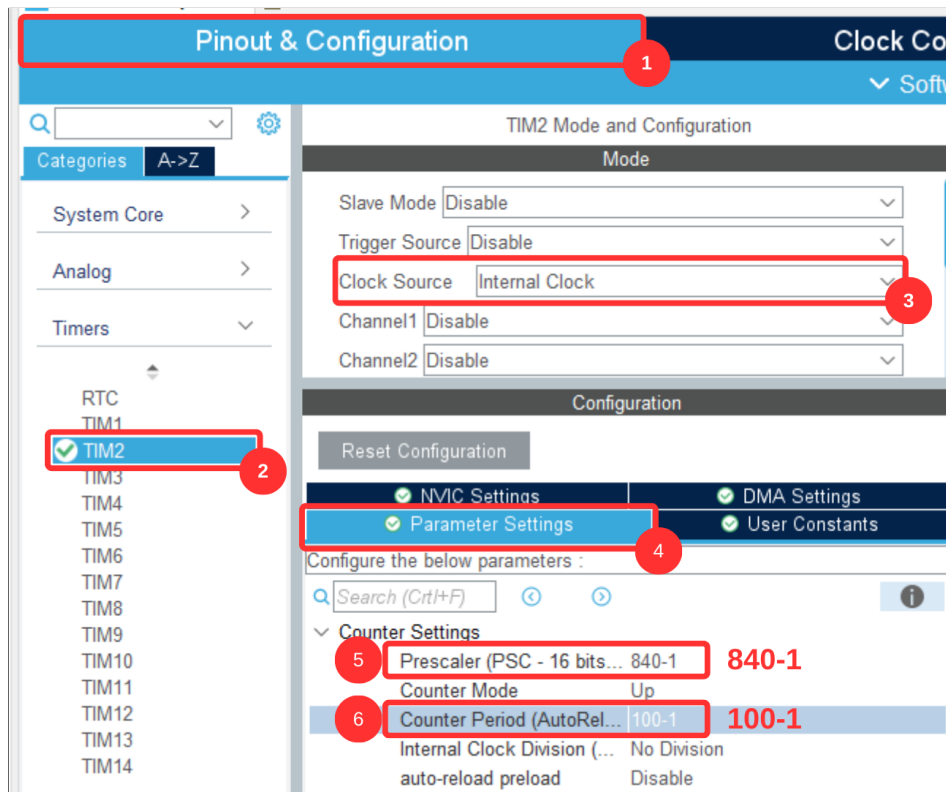
Bước 1: Tạo project mới được cấu hình như Lab 1.

Bước 2: Kiểm tra cấu hình Clock tại cửa sổ **Clock Configuration** (ở file .ioc). Kiểm tra cấu hình xung clock theo như hình dưới.



Hình 2.3: Clock source mặc định cho hệ thống

Bước 3: Cấu hình tham số TIM2 theo hình sau:

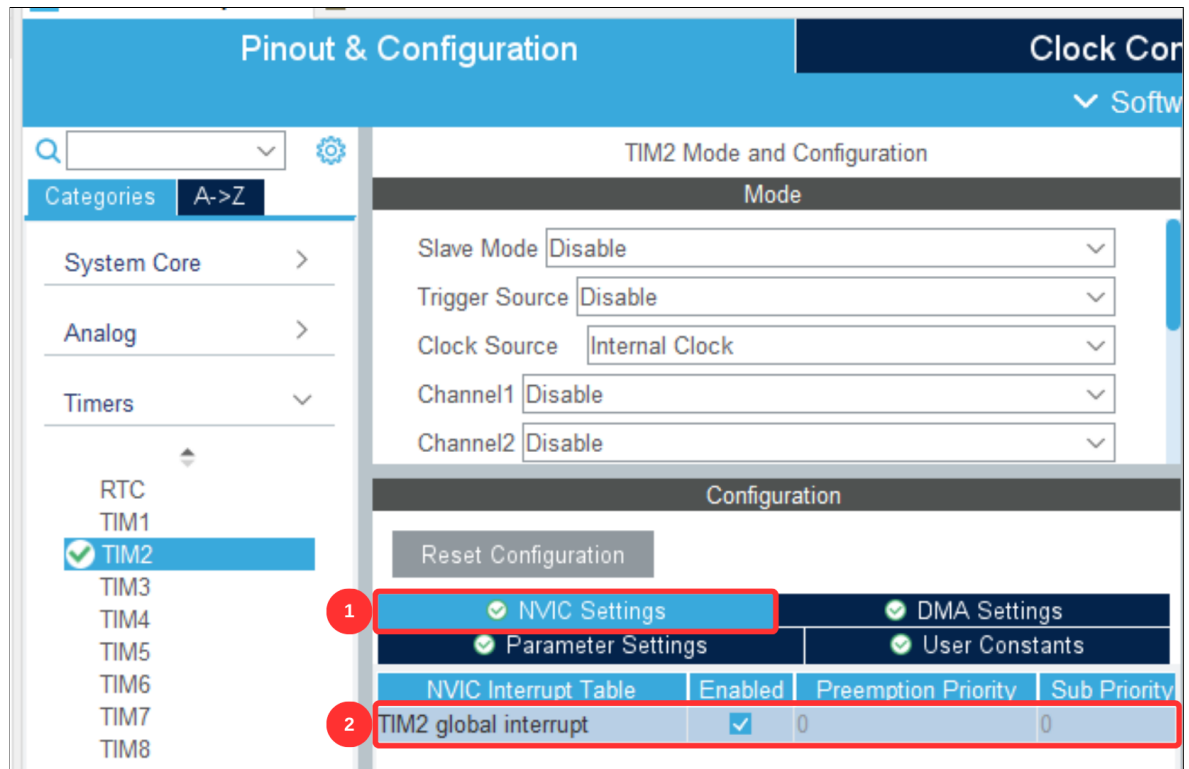


Hình 2.4: Cấu hình cho Timer 2

Chọn **Clock Source** cho timer 2 là **Internal Clock**. Cuối cùng cấu hình **Prescaler** là **840-1** và **Counter** là **100-1**. Những thông số này được giải thích như sau:

- Mục đích là tạo một timer với chu kỳ 1ms.
- Theo hướng dẫn sử dụng của chip STM32F407ZGT6, nguồn clock cấp cho module **TIM2** được kết nối với **APB1 Timer Clock**, mà theo chúng ta cấu hình ở hình 2.3 là 84MHz.
- Bằng cách đặt prescaler là 840-1, xung clock đầu vào của timer sẽ là $84\text{MHz}/(840-1+1) = 100\text{KHz}$.
- Hàm interrupt sẽ được gọi mỗi khi bộ đếm của timer đếm từ 0 tới 99, nghĩa là tần số lại tiếp tục được chia cho 100, trở thành **1kHz**, tương đương với chu kỳ **1ms**.

Bước 4: Kích hoạt timer interrupt tại cửa sổ **NVIC Settings** như sau:



Hình 2.5: Kích hoạt timer interrupt

Ngoài ra, ta có thể cấu hình để các file mã nguồn của các ngoại vi sẽ được hệ thống sinh ra theo cặp file *.c và *.h

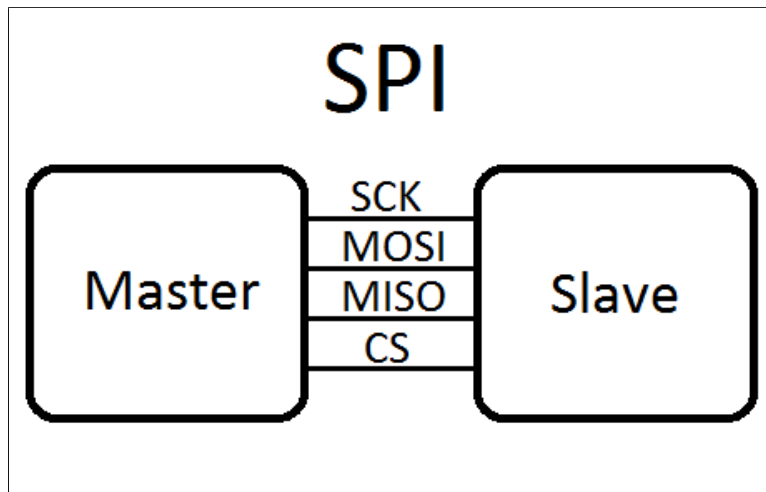
Cuối cùng lưu lại file cấu hình để hệ thống tự sinh code.

4 Cấu hình LED đồng hồ

LED đồng hồ trên kit thí nghiệm không được điều khiển trực tiếp từ chân MCU. Để tiết kiệm chân, LED đồng hồ được điều khiển thông qua 2 IC 74HC595. IC này có thể giao tiếp với vi điều khiển thông qua giao tiếp SPI.

4.1 Giao tiếp SPI

- SPI – Serial Peripheral Interface – còn gọi là giao diện ngoại vi nối tiếp, chuẩn đồng bộ nối truyền dữ liệu ở chế độ full-duplex (song công toàn phần). Nghĩa là tại 1 thời điểm có thể xảy ra đồng thời quá trình truyền và nhận.
- SPI là giao tiếp đồng bộ, bất cứ quá trình nào cũng đều được đồng bộ với xung clock sinh ra bởi thiết bị Master. Do đó, không cần phải lo lắng về tốc độ truyền dữ liệu.



Hình 2.6: Sơ đồ giao tiếp SPI

- Hoạt động của giao tiếp SPI: sử dụng 4 đường giao tiếp nên đôi khi được gọi là chuẩn truyền thông “ 4 dây”. 4 đường đó là :
 - SCK (Serial Clock): Thiết bị Master tạo xung tín hiệu SCK và cung cấp cho Slave. Xung này giữ nhịp cho giao tiếp SPI và truyền dữ liệu. Điều này giúp giảm lỗi và tăng tốc độ truyền.
 - MISO (Master Input Slave Output): Tín hiệu tạo bởi thiết bị Slave và nhận bởi thiết bị Master. Đường MISO phải được kết nối giữa thiết bị Master và Slave.
 - MOSI (Master Output Slave Input): Tín hiệu tạo bởi thiết bị Master và nhận bởi thiết bị Slave. Đường MOSI phải được kết nối giữa thiết bị Master và Slave.
 - CS (Chip Select) hoặc SS(Slave Select): Chọn Slave để giao tiếp. Master kéo chân CS xuống mức 0 (Low) để chọn Slave. Vì điều khiển có thể tạo chân CS bằng cách cấu hình 1 chân GPIO chế độ Output.

4.2 IC 74HC595

74HC595 là một thanh ghi dịch (shift register) hoạt động trên giao thức nối tiếp vào song song ra (Serial IN Parallel OUT), tức là nó có thể nhận dữ liệu đầu vào nối tiếp và điều khiển 8 chân đầu ra song song. Điều này rất tiện dụng khi không có đủ chân GPIO trên vi điều khiển hoặc vi xử lý để kiểm soát số lượng đầu ra cần thiết.

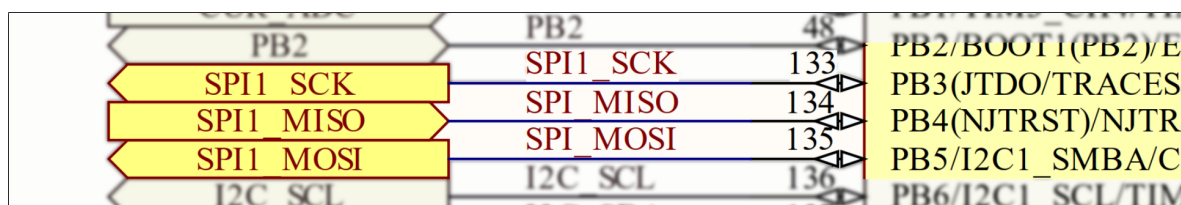
Đối với IC 74HC595, chúng ta cần quan tâm tới các chân **SH_CP**, **ST_CP**, **SDI**, **SDO**, **Q0-Q7**.

Với 2 IC thì ta có thể điều khiển được 16 output. Để điều khiển LED đồng hồ, ta cần 8 chân để điều khiển tín hiệu LED 7 đoạn, 4 chân để điều khiển chớp tắt 4 LED 7 đoạn, 1 chân để điều khiển dấu hai chấm. Như vậy chúng ta vẫn còn 3 output chưa được sử dụng, 3 output này sẽ được thiết kế để điều khiển LED6, LED7, LED8. Lưu ý: tất cả các chân tín hiệu kể trên đều tích cực mức thấp.

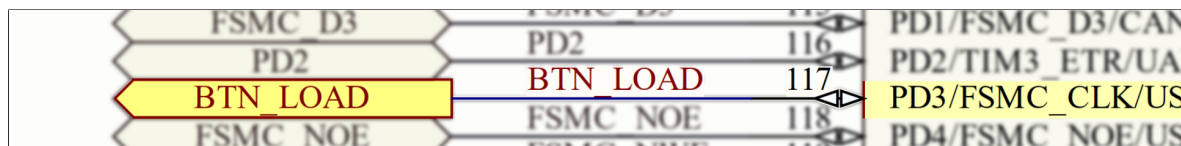
The diagram illustrates the LED display driver circuit. It consists of two 74HC595 shift registers, U14 and U15, connected to an SPI interface. U14 controls four LEDs (LED4-LED7) and four transistors (Q8-Q11) for the display segments. U15 controls the display segments (a-p) and the LD COLON signal. The display is a D8 F5463BH 8-digit 7-segment display. Power is supplied by 3V3 and GND rails with decoupling capacitors C70 and C71. Various resistors (R69-R84) are used for current limiting and signal conditioning.

4.3 Config chân cho vi điều khiển

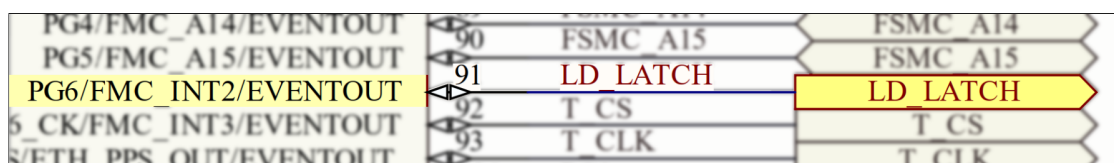
Trang 9



Hình 2.8: Sơ đồ nguyên lý của các chân SPI1_SCK, SPI1_MISO, SPI1_MOSI

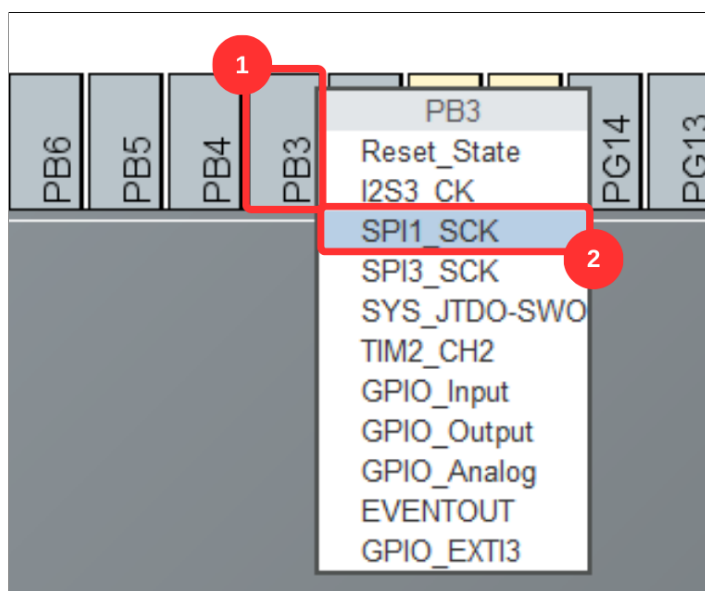


Hình 2.9: Sơ đồ nguyên lý của chân BTN_LOAD



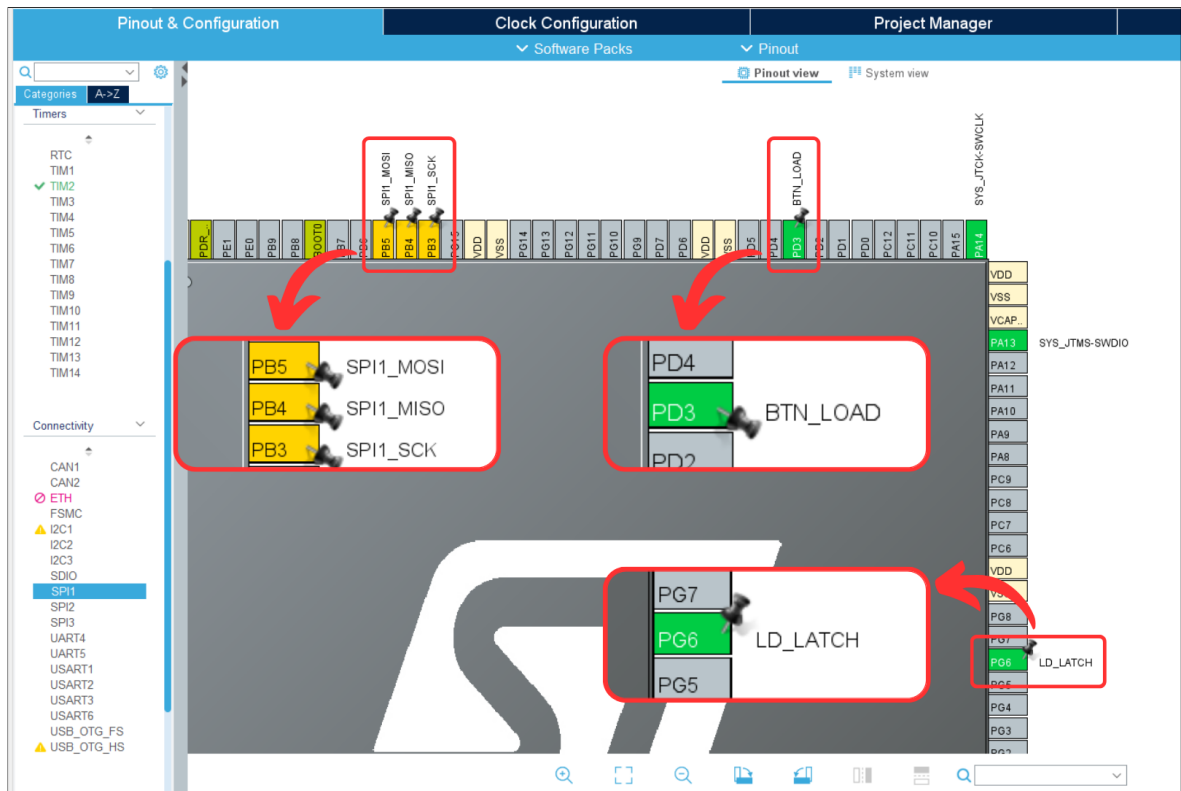
Hình 2.10: Sơ đồ nguyên lý của chân LD_LATCH

Ta thực hiện config các chân SPI1_SCK, SPI1_MISO, SPI1_MOSI bằng cách nhấp chuột phải vào chân tương ứng, sau đó chọn kiểu chân dựa theo schematic (hình 2.8). Hai chân BTN_LOAD và LD_LATCH sẽ được config thành chân output.



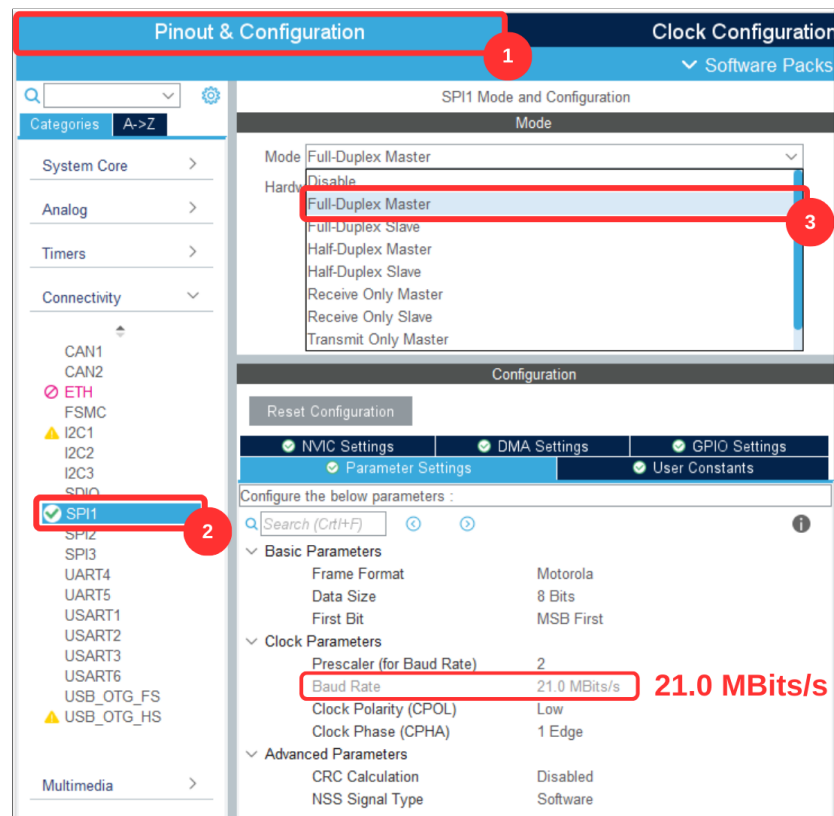
Hình 2.11: Config các chân SPI1_SCK, SPI1_MISO, SPI1_MOSI

Sau khi config, ta có được Pinout View như sau:



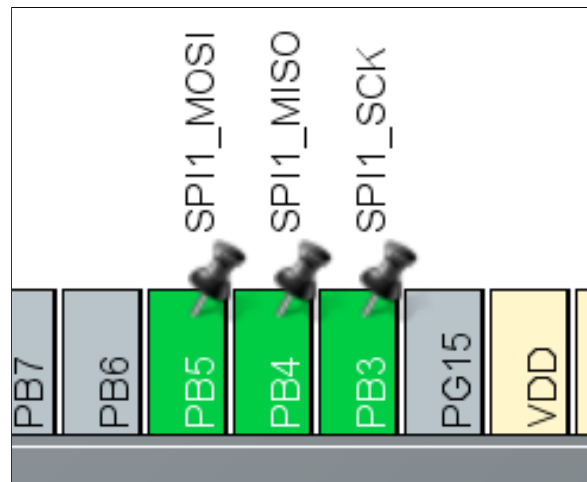
Hình 2.12: Các chân cần config

Để vi điều khiển giao tiếp với LED đồng hồ, ta cần config SPI1 như hình sau:



Hình 2.13: Config SPI

Sau khi config SPI1, ta sẽ có Pinout View như sau:



Hình 2.14: Sau khi config SPI1

5 Hướng dẫn lập trình

Các file **software_timer** và **LED7_seg** có thể được copy từ project mẫu **Bai2_Timer_7seg**.

```
1 #ifndef INC_SOFTWARE_TIMER_H_
2 #define INC_SOFTWARE_TIMER_H_
3
4 #include "tim.h"
5 #include "software_timer.h"
6 #include "LED_7seg.h"
7
8 extern uint16_t flag_timer2;
9
10 void timer_init();
11 void setTimer2(uint16_t duration);
12
13 #endif /* INC_SOFTWARE_TIMER_H_ */
```

Program 2.1: software_timer.h

```

1 #include "software_timer.h"
2
3 #define TIMER_CYCLE_2 1
4
5 uint16_t flag_timer2 = 0;
6 uint16_t timer2_counter = 0;
7 uint16_t timer2_MUL = 0;
8
9 void timer_init(){
10     HAL_TIM_Base_Start_IT(&htim2);
11 }
12
13 void setTimer2(uint16_t duration){
14     timer2_MUL = duration/TIMER_CYCLE_2;
15     timer2_counter = timer2_MUL;
16     flag_timer2 = 0;
17 }
18
19 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
20 {
21     if(htim->Instance == TIM2){
22         if(timer2_counter > 0){
23             timer2_counter--;
24             if(timer2_counter == 0) {
25                 flag_timer2 = 1;
26                 timer2_counter = timer2_MUL;
27             }
28         }
29         // 1ms interrupt here
30         LED7_Scan();
31     }
32 }

```

Program 2.2: software_timer.c

Ý tưởng của software timer là từ timer interrupt 1ms, ta sẽ dùng các biến đếm để có thể tạo ra các timer mềm với tần số thấp hơn.

- Hàm **HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef*htim)**: là hàm interrupt service routine (isr) được gọi mỗi 1ms. Trong hàm này, các counter

của software timer sẽ được đếm lùi, và khi hết thời gian một tín hiệu `flag_timer` sẽ được bật để chương trình chính nhận biết.

- Hàm **timer_init**: sẽ được gọi để khởi tạo timer của vi điều khiển.
- Hàm **setTimer2**: sẽ khởi tạo tần số của software timer 2. Nếu như cần sử dụng nhiều software timer hơn thì có thể tạo thêm các biến đếm và hàm tương tự.

```
1 #ifndef INC_LED7SEG_H_
2 #define INC_LED7SEG_H_
3
4 #include "spi.h"
5
6 void LED7_init();
7 void LED7_Scan();
8 void LED7_SetDigit(int num, int position, uint8_t show_dot)
9     ;
10 void LED7_SetColon(uint8_t status);
11
12 void LED_On(uint8_t index);
13 void LED_Off(uint8_t index);
14 #endif /* INC_LED7SEG_H_ */
```

Program 2.3: LED7_seg.h

- **LED7_init**: hàm khởi tạo LED đồng hồ
- **LED7_Scan**: hàm quét LED. Hiện tại hàm này được gọi trong timer interrupt. Có nghĩa tần số quét hiện là $1\text{KHz}/4 = 250\text{Hz}$. Để thử nghiệm thay đổi tần số quét thì có thể dùng software timer và thay đổi vị trí đặt hàm này.
- **LED7_SetDigit**: hàm hiển thị số trên LED, tham số đầu tiên là số muốn hiển thị, tham số thứ 2 là vị trí của LED muốn hiển thị số đó (0-4), tham số cuối cùng thể hiện rằng dấu chấm tại LED 7 đoạn đó có được hiển thị hay không.
- **LED7_SetColon**: hàm hiển thị hoặc tắt dấu hai chấm.
- **LED_On**: hàm bật các LED 6,7,8; tham số đầu vào là index của các LED muốn bật.
- **LED_Off**: hàm tắt các LED 6,7,8; tham số đầu vào là index của các LED muốn tắt.

```

1 int main(void)
2 {
3     //...
4     /* USER CODE BEGIN 2 */
5     system_init();
6     LED7_SetColon(1);
7     /* USER CODE END 2 */
8
9     /* Infinite loop */
10    /* USER CODE BEGIN WHILE */
11    while (1)
12    {
13        while(!flag_timer2);
14        flag_timer2 = 0;
15        // main task, every 50ms
16        test_LEDDebug();
17        test_LEDY0();
18        test_LEDY1();
19        test_7seg();
20        /* USER CODE END WHILE */
21
22        /* USER CODE BEGIN 3 */
23    }
24    /* USER CODE END 3 */
25 }
26
27 /* USER CODE BEGIN 4 */
28 void system_init(){
29     HAL_GPIO_WritePin(OUTPUT_Y0_GPIO_Port, OUTPUT_Y0_Pin,
30     0);
31     HAL_GPIO_WritePin(OUTPUT_Y1_GPIO_Port, OUTPUT_Y1_Pin,
32     0);
33     HAL_GPIO_WritePin(DEBUG_LED_GPIO_Port, DEBUG_LED_Pin,
34     0);
35     timer_init();
36     LED7_init();
37     setTimer2(50);
38 }

```

Program 2.4: hàm main

Để rõ ràng cho việc lập trình, các code khởi tạo sẽ được để trong hàm **system_init** và hàm này sẽ được gọi trước vòng **while** trong hàm **main**.

Câu lệnh **SetTimer2(50)** sẽ khởi tạo một software timer có chu kì là 50ms. Trong công nghiệp, 50ms thường là chu kì ổn định cho các việc đọc input, xử lí và xuất tín hiệu output.

Trong vòng lặp vô tận while, ta sẽ chờ cho đến khi tín hiệu **flag_timer2** được bật (mỗi 50ms) sau đó sẽ gọi các hàm đọc input, xử lý hoặc xuất tín hiệu output.

```
1 uint8_t count_LED_debug = 0;
2 uint8_t count_LED_Y0 = 0;
3 uint8_t count_LED_Y1 = 0;
4
5 void test_LEDDebug(){
6     count_LED_debug = (count_LED_debug + 1)%20;
7     if(count_LED_debug == 0){
8         HAL_GPIO_TogglePin(DEBUG_LED_GPIO_Port , DEBUG_LED_Pin);
9     }
10 }
11
12 void test_LEDY0(){
13     count_LED_Y0 = (count_LED_Y0+ 1)%100;
14     if(count_LED_Y0 > 40){
15         HAL_GPIO_WritePin(OUTPUT_Y0_GPIO_Port , OUTPUT_Y0_Pin ,
16             1);
17     } else {
18         HAL_GPIO_WritePin(OUTPUT_Y0_GPIO_Port , OUTPUT_Y0_Pin ,
19             0);
20     }
21 }
22
23 void test_LEDY1(){
24     count_LED_Y1 = (count_LED_Y1+ 1)%40;
25     if(count_LED_Y1 > 10){
26         HAL_GPIO_WritePin(OUTPUT_Y1_GPIO_Port , OUTPUT_Y1_Pin ,
27             0);
28     }
```

```

25 } else {
26     HAL_GPIO_WritePin(OUTPUT_Y0_GPIO_Port, OUTPUT_Y1_Pin,
27         1);
28 }
29
30 void test_7seg(){
31     //write number1 at LED index 0 (not show dot)
32     LED7_SetDigit(1, 0, 0);
33     LED7_SetDigit(5, 1, 0);
34     LED7_SetDigit(4, 2, 0);
35     LED7_SetDigit(7, 3, 0);
36 }

```

Program 2.5: Các hàm test

Các hàm trên được tạo với mục đích kiểm tra các module trên kit thí nghiệm và làm quen với cách sử dụng các hàm trong thư viện. Các hàm này sẽ được gọi trong vòng while mỗi 50ms.

Như vậy đối với hàm **test_LEDDebug** sau 50ms thì biến **count_LED_debug** được tăng lên và cứ sau 20 lần gọi hàm ($20 \times 50 = 1000\text{ms} = 1\text{s}$) thì biến **count_LED_debug** sẽ bằng 0 và tác vụ toggle LED sẽ được thực thi. Như vậy LED3 sẽ chớp tắt mỗi 1s.

Với cách suy luận tương tự thì hàm **test_LEDY0** và **test_LEDY1** sẽ tạo ra hiệu ứng chớp tắt LED với thời gian khác nhau.

Hàm **test_7seg** sẽ hiển thị các giá trị lên LED đồng hồ, cụ thể LED đồng hồ sẽ hiển thị số "15:47".

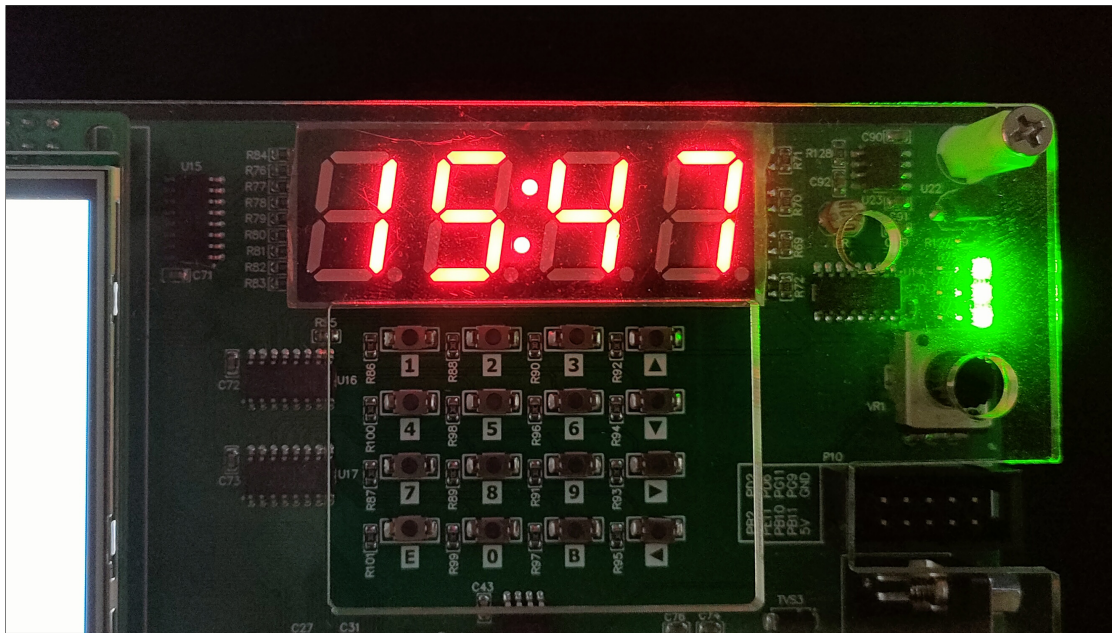
Hình ảnh kết quả trên Kit thí nghiệm:

6 Bài tập và Báo cáo

6.1 Bài tập 1

Dùng software timer và tham khảo các hàm ở trên để tạo hiệu ứng trên các LED:

- **DEBUG_LED** chớp tắt mỗi 2s.



Hình 2.15: Kết quả hiện thực trên Kit thí nghiệm

- LED_Y0 sáng trong vòng 2s và tắt trong 4s.
- LED_Y1 sáng trong vòng 5s và tắt trong 1s.

6.2 Bài tập 2

Hiện thực lại bài tập về đèn giao thông ở Lab 1 sử dụng timer.

6.3 Bài tập 3

Thay đổi tần số quét của LED đồng hồ thành 1Hz, 25Hz, 100Hz.

6.4 Bài tập 4

Dùng LED đồng hồ để mô phỏng đồng hồ kĩ thuật số, 2 số đầu tiên sẽ hiển thị giờ, 2 số tiếp theo sẽ hiển thị phút. Thời gian sẽ thay đổi như thời gian thực. Dấu 2 chấm ở giữa đồng hồ sẽ chớp tắt với tần số 2Hz.

6.5 Bài tập 5

Hiển thị 4 số khác nhau trên LED đồng hồ, tạo hiệu ứng các số này sẽ được dịch qua phải mỗi 1s.